

16.04.2026

Сравнение подходов Go, .NET и Java к Profile-Guided Optimization



Егоров Дмитрий

Разработчик сервисов
информационной безопасности

Почему я об этом рассказываю ?

У кого-то хобби футбол, а у меня
JIT, поэтому:

Серийный спикер про JIT

- a) DotNext
- b) SpbDotNet
- c) DotNetRu курс!



Это я на DUMP SPB в 2025 году

Что такое profile guided optimization?

Выполнение общего пайплайна оптимизаций со статистикой о профиле использования программы



Почему доклад именно об этом ?

1. PGO - фундаментальная штука, которая сказывается на всей работе оптимизирующего компилятора
2. Тренды развития языков

Мы будем использовать псевдоязык, asm будет мало

G_M000_IG01: ;; offset=0x0000

```
stp    fp, lr, [sp, #-0x40]!
mov    fp, sp
str    wzr, [fp, #0x30]
str    wzr, [fp, #0x2C]
str    xzr, [fp, #0x20]
str    xzr, [fp, #0x10]
str    x0, [fp, #0x38]
str    w1, [fp, #0x34]
```

G_M000_IG02: ;; offset=0x0020

```
mov    w0, #0x3E8
str    w0, [fp, #0x18]
str    wzr, [fp, #0x30]
str    wzr, [fp, #0x2C]
b      G_M000_IG04
```

G_M000_IG03: ;; offset=0x0034

```
movz   x0, #0x5D78
movk   x0, #0x80C LSL #16
movk   x0, #1 LSL #32
bl     CORINFO_HELP_COUNTPROFILE32
ldr     x0, [fp, #0x38]
str     x0, [fp, #0x20]
ldr     x0, [fp, #0x20]
movz   x1, #0x5D80
movk   x1, #0x80C LSL #16
movk   x1, #1 LSL #32
bl     CORINFO_HELP_CLASSPROFILE32
ldr     x0, [fp, #0x20]
str     x0, [fp, #0x10]
ldr     x0, [fp, #0x10]
ldr     w1, [fp, #0x2C]
movz   x11, #24
movk   x11, #0x6E6 LSL #16
movk   x11, #1 LSL #32
ldr     xip0, [x11]
blr     xip0
ldr     w1, [fp, #0x30]
add     w0, w0, w1
str     w0, [fp, #0x30]
ldr     w0, [fp, #0x2C]
add     w0, w0, #1
str     w0, [fp, #0x2C]
```

G_M000_IG04: ;; offset=0x009C

```
ldr     w0, [fp, #0x18]
sub     w0, w0, #1
str     w0, [fp, #0x18]
ldr     w0, [fp, #0x18]
cmp     w0, #0
bgt     G_M000_IG06
```

G_M000_IG05: ;; offset=0x00B4

```
add     x0, fp, #24
mov     w1, #20
bl      CORINFO_HELP_PATCHPOINT
```

G_M000_IG06: ;; offset=0x00C0

```
ldr     w0, [fp, #0x2C]
ldr     w1, [fp, #0x34]
cmp     w0, w1
blt     G_M000_IG03
movz    x0, #0x5E88
movk    x0, #0x80C LSL #16
movk    x0, #1 LSL #32
bl      CORINFO_HELP_COUNTPROFILE32
ldr     w0, [fp, #0x30]
```

G_M000_IG07: ;; offset=0x00E4

```
ldp     fp, lr, [sp], #0x40
ret     lr
```

Мы будем использовать псевдоязык, asm будет мало

```
int Run(IWorker worker, int n)
{
    int sum = 0;
    int i = 0;
    object tmpObj = null;
    object tmpForCall = null;
    int patchCountdown = 1000;

    goto LOOP_TEST;

LOOP_BODY:
    CORINFO_HELP_COUNTPROFILE32(profileBody);

    tmpObj = worker;
    CORINFO_HELP_CLASSPROFILE32(profileClass, tmpObj);

    tmpForCall = tmpObj;
    int ret = tmpForCall.Work(i);    // косвенный вызов через blr

    sum = sum + ret;
    i = i + 1;

LOOP_TEST:
    patchCountdown = patchCountdown - 1;
    if (patchCountdown <= 0)
    {
        CORINFO_HELP_PATCHPOINT(&patchCountdown, 20);
    }

    if (i < n)
        goto LOOP_BODY;

    CORINFO_HELP_COUNTPROFILE32(profileExit);
    return sum;
}
```

Что я хочу донести в докладе ?

- 1) Что такое PGO, зачем оно нужно
- 2) Какие оптимизации становятся возможны благодаря PGO
- 3) Как и когда статистика может собираться и применяться
- 4) Как PGO выглядит на уровне результата компиляции
- 5) Кто круче использует PGO)

Результаты внедрения PGO

PGO IMPLEMENTER

"This will definitely improve performance" -- breaks production

"Just need to tweak a few more flags" -- 500 compiler errors

"Profile-guided optimization is the future" -- can't profile real workloads

"Trust me, I know what I'm doing" -- hasn't read the documentation

"Just one more build cycle" -- said 6 hours ago



"We'll see 20% speedup easy" --
0.1% improvement after 3 months

"It's not my code, it's the compiler" -- forgot to enable optimizations

"PGO will solve all our problems" -- introduces 17 new ones



В среднем 5-30% улучшения

Результаты внедрения PGO

Benchmark	Old time/op	New time/op	Delta
Template	71.0ms ± 2%	68.3ms ± 2%	-3.90%
Unicode	71.8ms ± 2%	66.8ms ± 2%	-6.90%
GoTypes	444ms ± 1%	428ms ± 1%	-3.53%
Compiler	48.9ms ± 3%	45.6ms ± 3%	-6.81%
SSA	3.25s ± 2%	3.09s ± 1%	-5.03%
Flate	44.0ms ± 2%	42.3ms ± 2%	-3.72%
GoParser	76.7ms ± 1%	73.5ms ± 1%	-4.15%
Reflect	172ms ± 1%	165ms ± 1%	-4.13%
Tar	63.1ms ± 1%	60.4ms ± 2%	-4.24%
XML	83.2ms ± 2%	79.2ms ± 2%	-4.79%
Geo mean	127ms	121ms	-4.73%

495596: cmd/compile: build compiler with PGO | <https://go-review.googlesource.com/c/go/+495596>

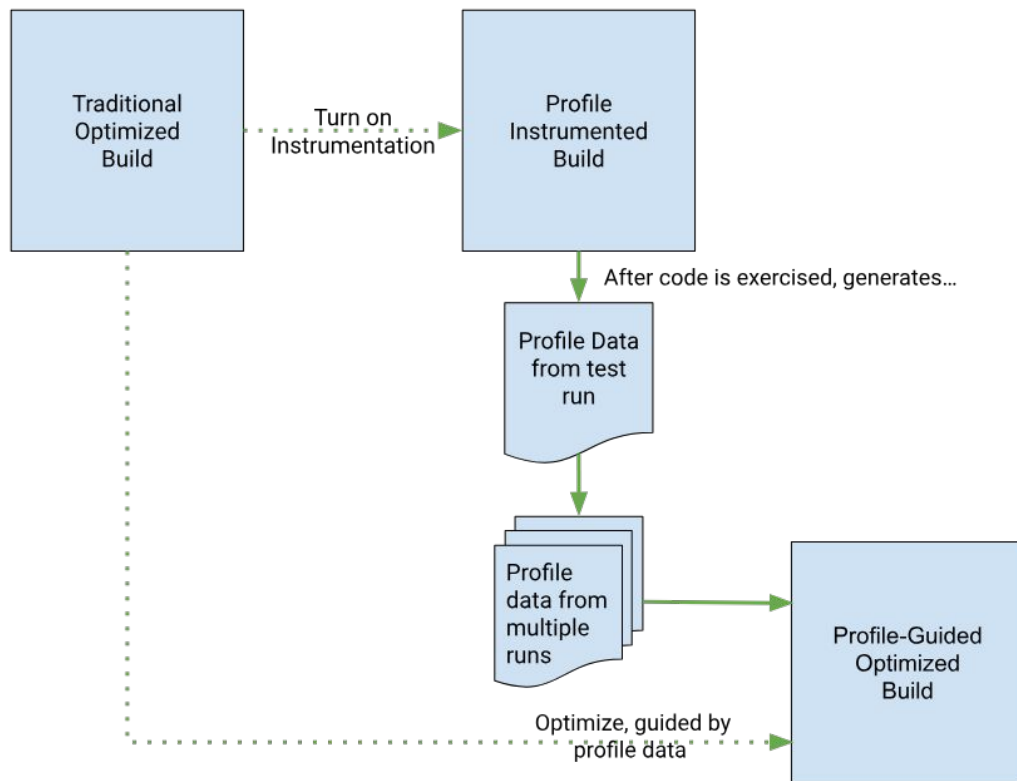
Глава 1

Что такое PGO?

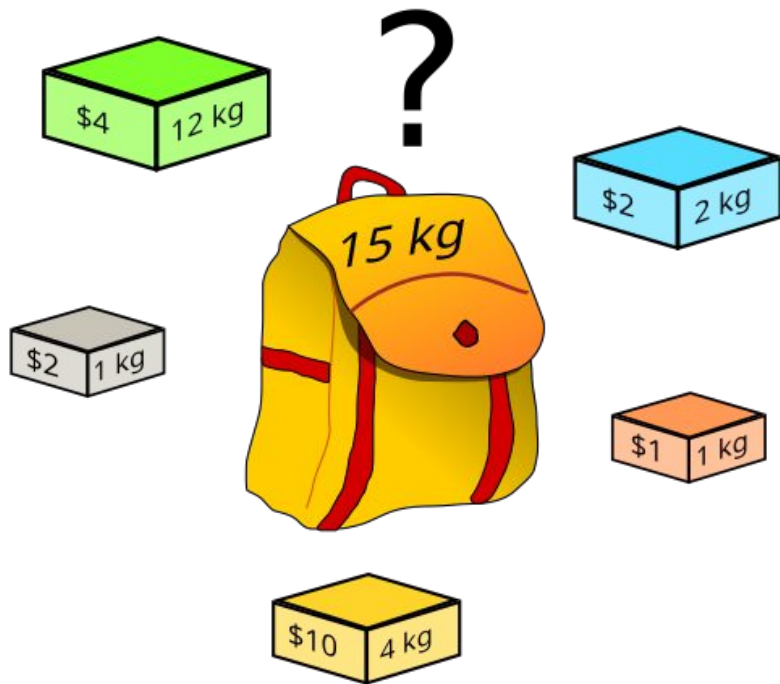
Зачем существует ?

Отличия .NET, JAVA, GO в работе

Что такое PGO ?



А нельзя без PGO это делать ?

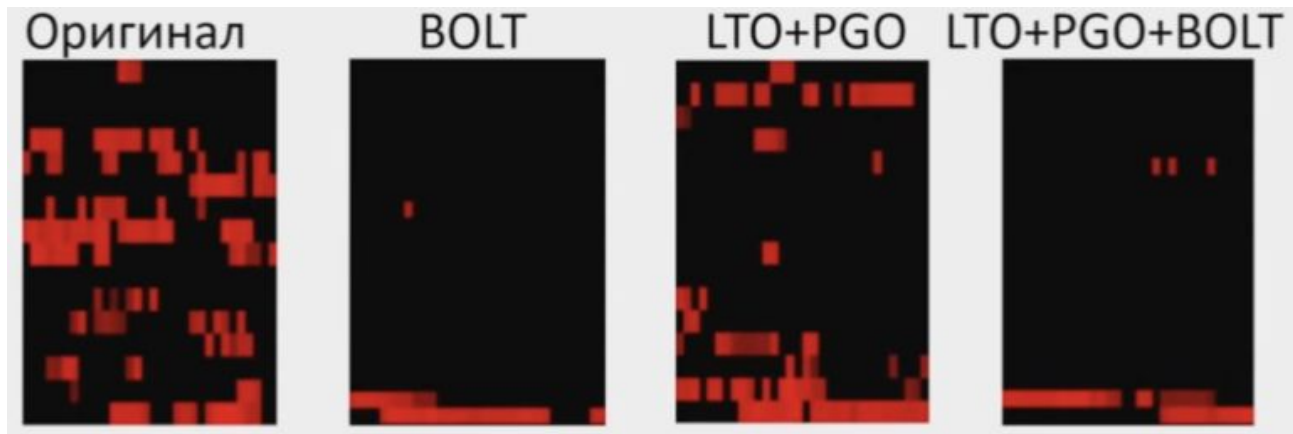


А нельзя без PGO это делать ?

Еще проблемы без PGO

- JIT не может компилировать долго
- Многие методы холодные 90%, их нет смысла компилировать

А нельзя без PGO это делать ?

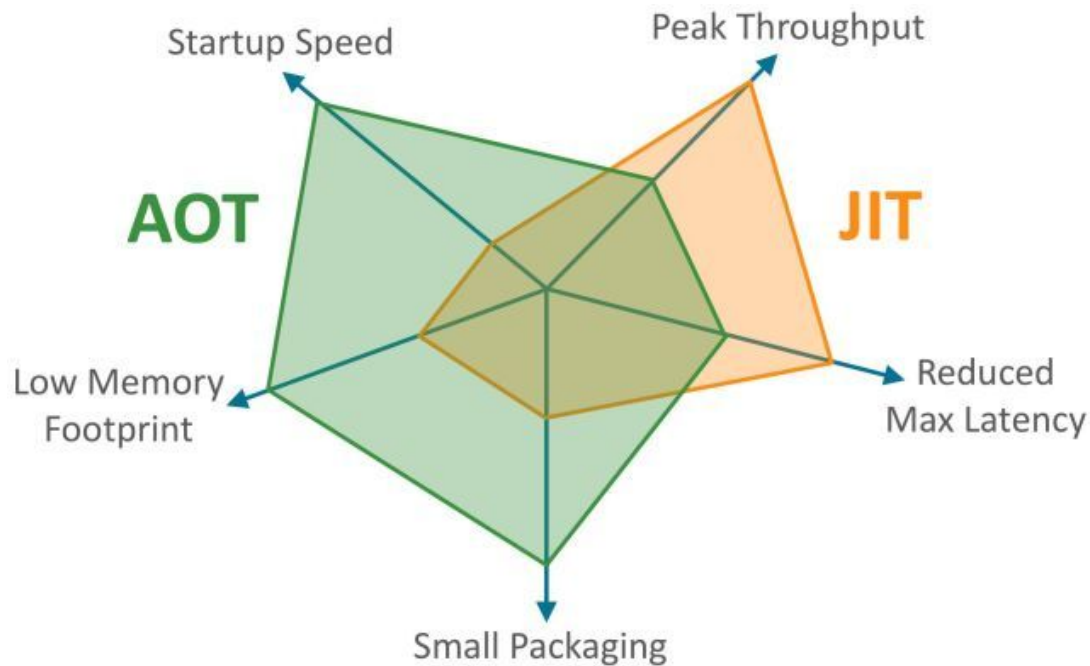


Локализация обращения к памяти nginx



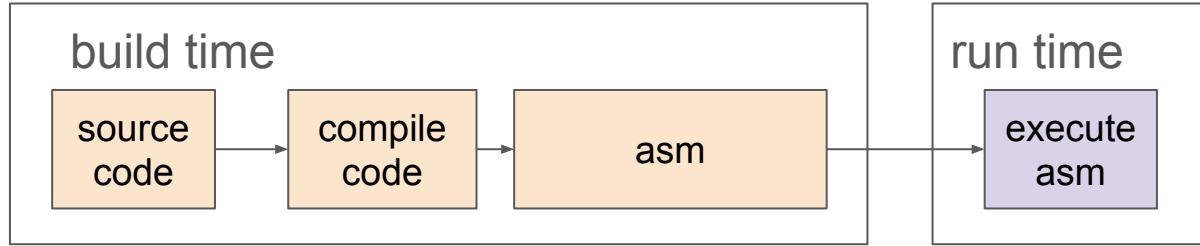
Василий Леоненко — BOLT

Отличия .NET, JAVA, GO

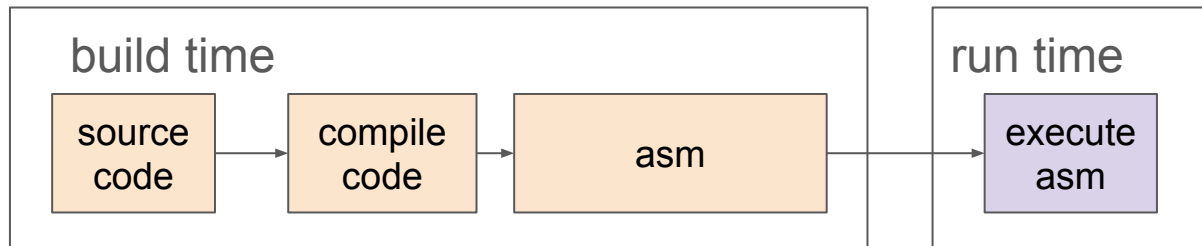


Андрей Порожняков «Native AOT.
Возможности и ограничения»

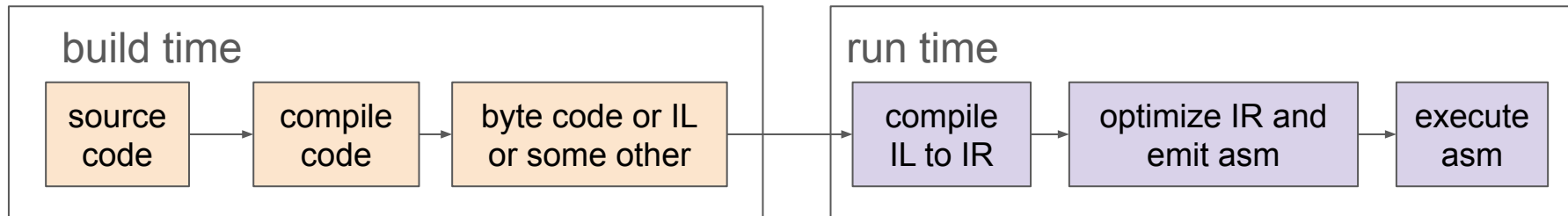
Aot



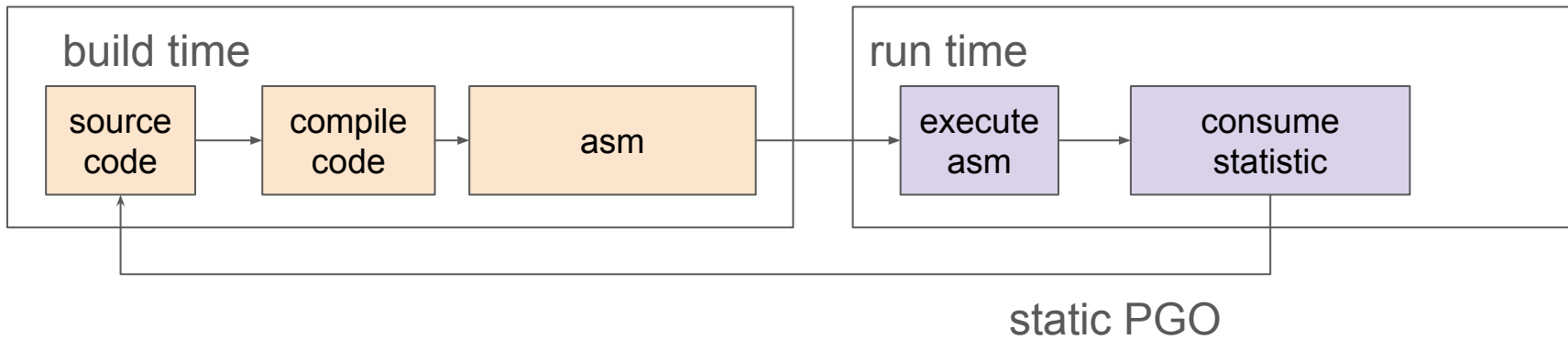
Aot



JIT



Aot



Билд и сборка profile

```
go build -o app . && GODEBUG=asyncpreemptoff=1 ./app profile >/dev/null
```

Билд с и без

```
&& go build -a -pgo=off -o app-off .
```

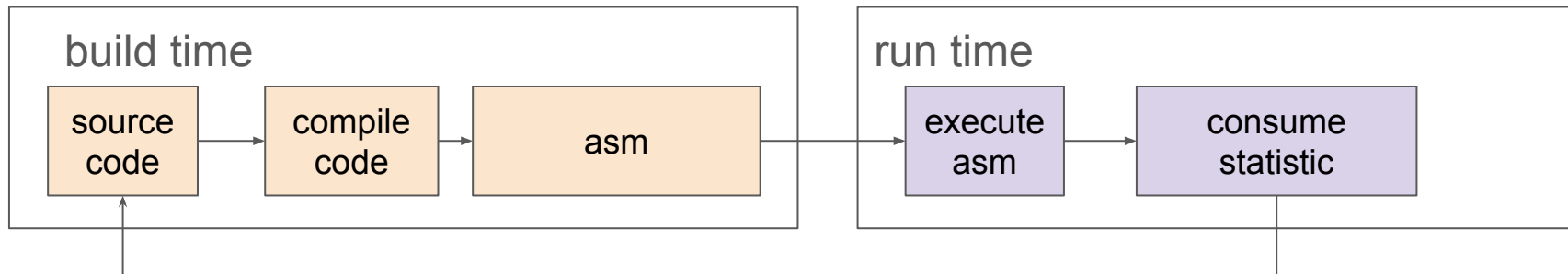
```
&& go build -a -pgo=cpu.pprof -o app-on .
```

Вывод результата

```
&& echo '=== OFF ===' && go tool objdump -s 'main\.runLoop|main\.run|main\.Hot\Work|main\.Cold\Work' ./app-off
```

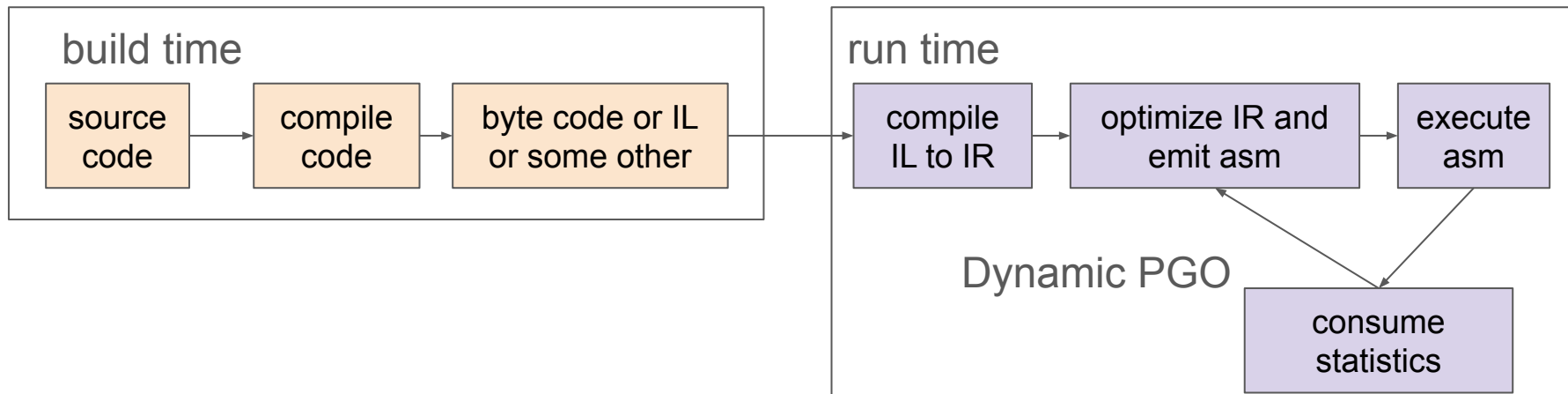
```
&& echo '=== ON ===' && go tool objdump -s 'main\.runLoop|main\.run|main\.Hot\Work|main\.Cold\Work' ./app-on
```

Aot

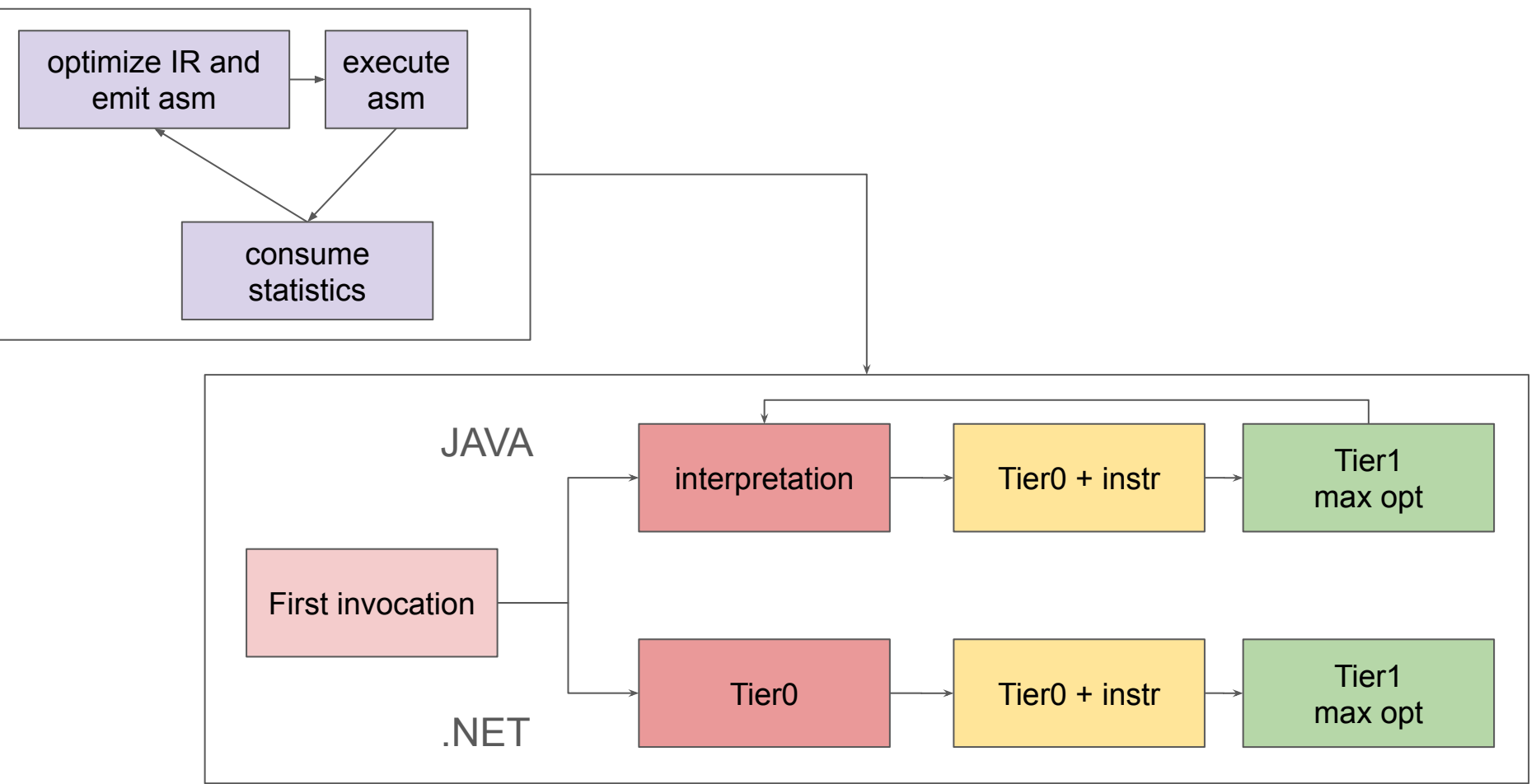


static PGO

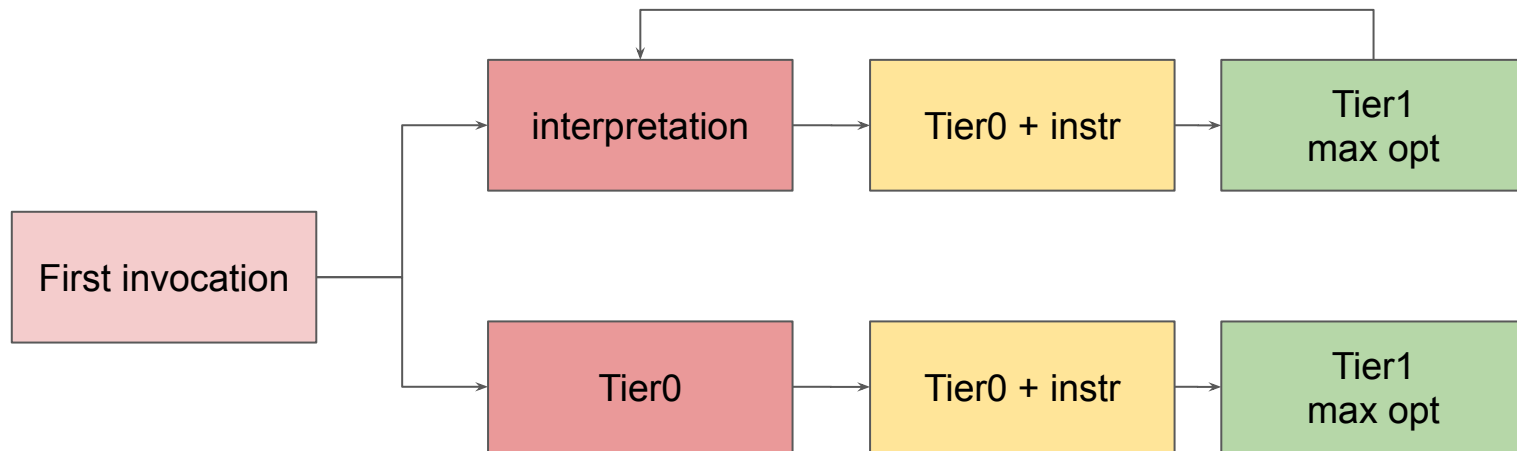
JIT



Dynamic PGO



Проблемы/особенности



- 1) .NET после Tier1 уже не сможет изменить профиль компиляции
- 2) JAVA может сделать деоптимизацию, но не делает это бесконечно

Пример 1 - Первый раунд (JAVA)



```
static int workload(int[] data, boolean hotProfile) {  
    int total = 0;  
  
    Worker w = pickWorker(hotProfile);  
  
    for (int i = 0; i < data.length; i++) {  
        total += w.work(data[i]);  
    }  
  
    return total;  
}
```



```
int workload(int[] data, boolean hotProfile) {  
    int total = 0;  
  
    Worker w = pickWorker(hotProfile);  
  
    if (w.getClass() != Hot.class) {  
        uncommon_trap_and_deopt();  
    }  
  
    for (int i = 0; i < data.length; i++) {  
        int x = data[i];  
  
        // inlined Main$Hot.work(x)  
        int y = x + 1;  
        int a = y * 3 - 7;  
        int b = a + x * 5 + 11;  
        int c = b ^ (a + x);  
        int d = c + b - a + x;  
        int e = d * 3 - c + 17;  
        if (e < 0) e = -e;  
        total += e + a;  
    }  
  
    return total;  
}
```

Пример 1 - Второй раунд (JAVA)



```
static int workload(int[] data, boolean hotProfile) {  
    int total = 0;  
  
    Worker w = pickWorker(hotProfile);  
  
    for (int i = 0; i < data.length; i++) {  
        total += w.work(data[i]);  
    }  
  
    return total;  
}
```



```
int workload(int[] data, boolean hotProfile) {  
    int total = 0;  
  
    Worker w = pickWorker(hotProfile);  
  
    if (w.getClass() == Cold.class) {  
        ...  
        return total;  
    }  
  
    if (w.getClass() == Hot.class) {  
        ...  
        return total;  
    }  
  
    uncommon_trap_or_generic_fallback();  
  
    return total;  
}
```

Пример 2 - Первый раунд (JAVA)

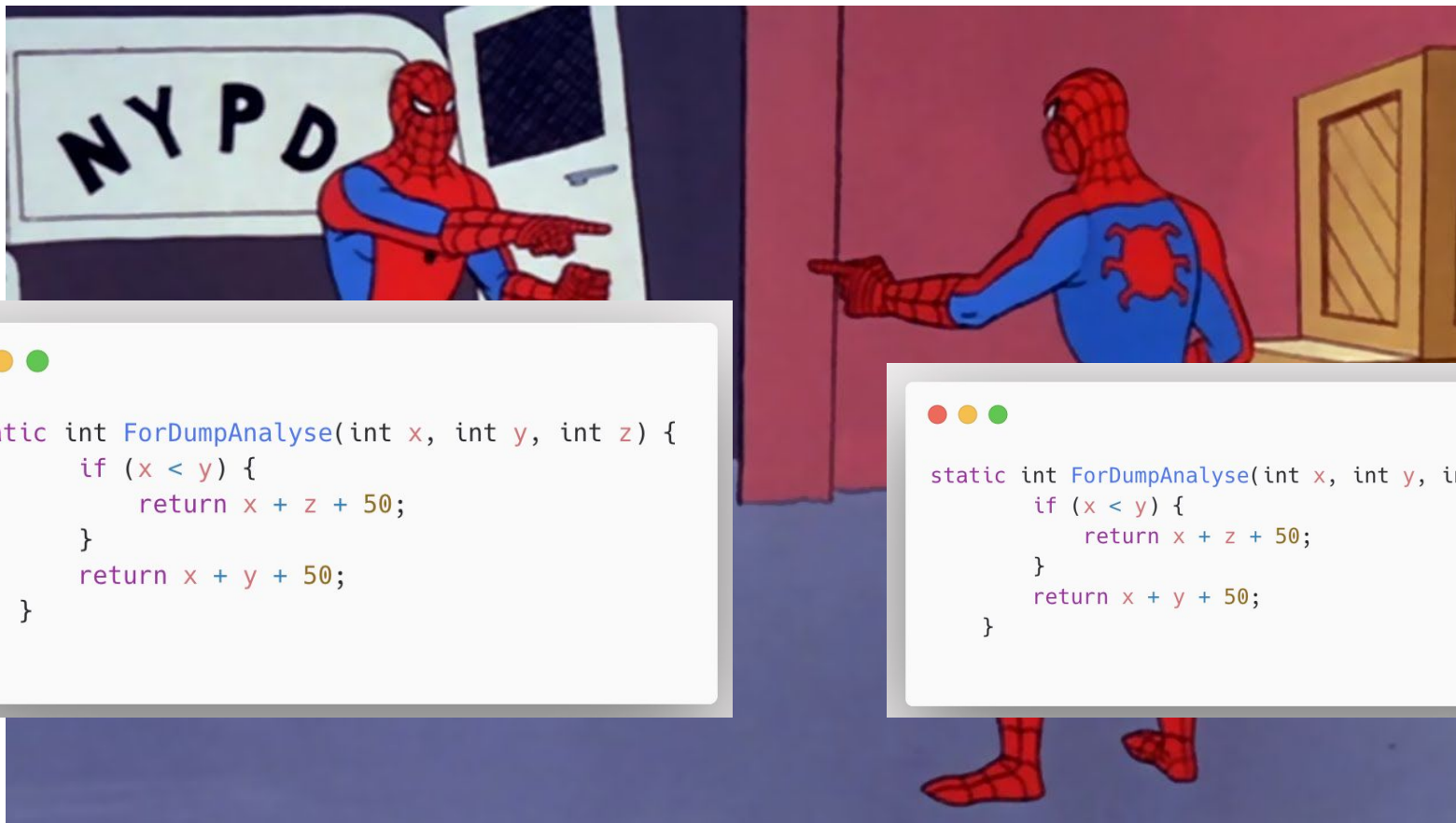


```
static int ForDumpAnalyse(int x, int y, int z) {  
    if (x < y) {  
        return x + z + 50;  
    }  
    return x + y + 50;  
}
```



```
int ForDumpAnalyse(int x, int y, int z) {  
    if (x >= y) {  
        uncommon_trap_and_reinterpret();  
    }  
    return x + z + 50;  
}
```

Пример 2 - Второй раунд (JAVA)



```
static int ForDumpAnalyse(int x, int y, int z) {  
    if (x < y) {  
        return x + z + 50;  
    }  
    return x + y + 50;  
}
```

```
static int ForDumpAnalyse(int x, int y, int z) {  
    if (x < y) {  
        return x + z + 50;  
    }  
    return x + y + 50;  
}
```

A .NET



```
static int ForDumpAnalyse(int x, int y, int z) {  
    if (x < y) {  
        return x + z + 50;  
    }  
    return x + y + 50;  
}
```



```
Program:ForDumpAnalyse(int,int,int):int:this (FullOpts):  
    lea     eax, [rsi+rdx+0x32]  
    lea     ecx, [rsi+rcx+0x32]  
    cmp     esi, edx  
    cmovl   eax, ecx  
    ret
```

Реклама курса



```
static int ForDumpAnaly
    if (x < y) {
        return x +
    }
    return x + y +
}
```

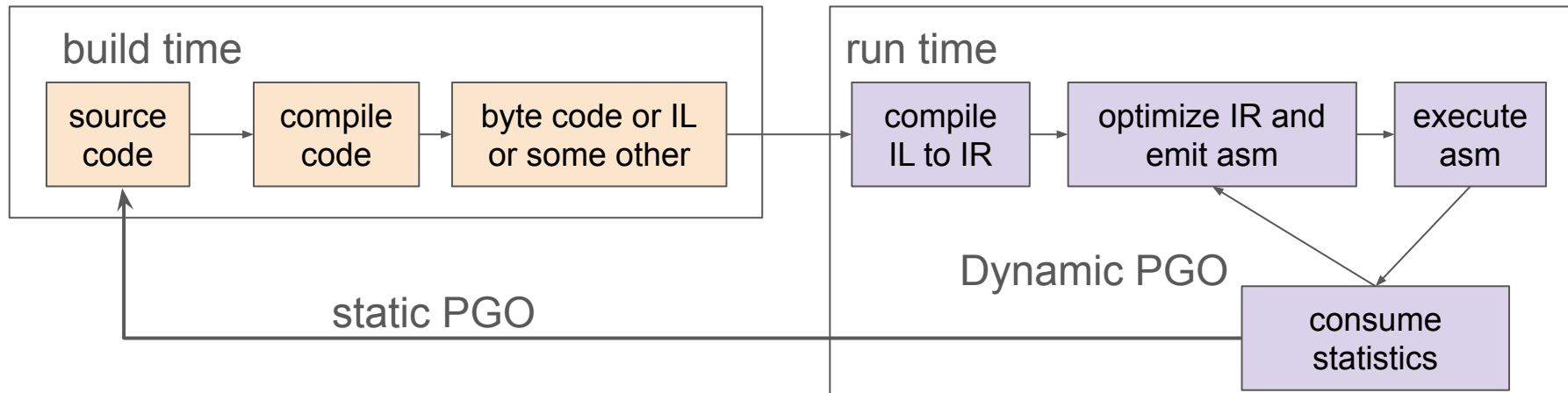


```
t):int:this (Full0pts):
2]
2]
```

В чем главная проблема этого подхода?

- 1) Долгий старт
- 2) Делаем на каждый старт одно и тоже
- 3) Первоначальная сборка не учитывает профиль, а физически он же существует

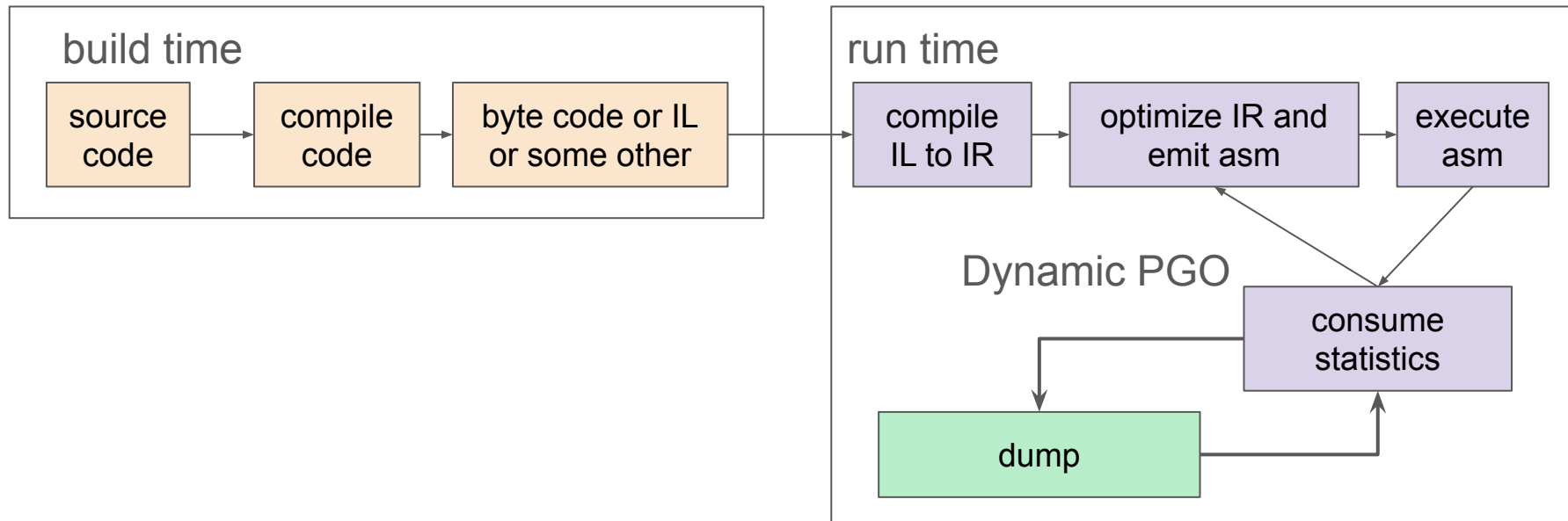
Мысль 1: А что нельзя профиль отдать в билд? (.NET)



Just 30 calls and that data will be there forever for all possible callsites(

Поэтому static требует отдельный режим сборки и выполнения, который мы не будем на проде использовать

Мысль 2: А может из dump подниматься ? (JAVA)



Coordinated Restore at Checkpoint в JAVA пытается решить проблему перезапуска, но это очень чувствительная технология

Coordinated Restore at Checkpoint

```
package my.app;

import org.crac.Context;
import org.crac.Core;
import org.crac.Resource;

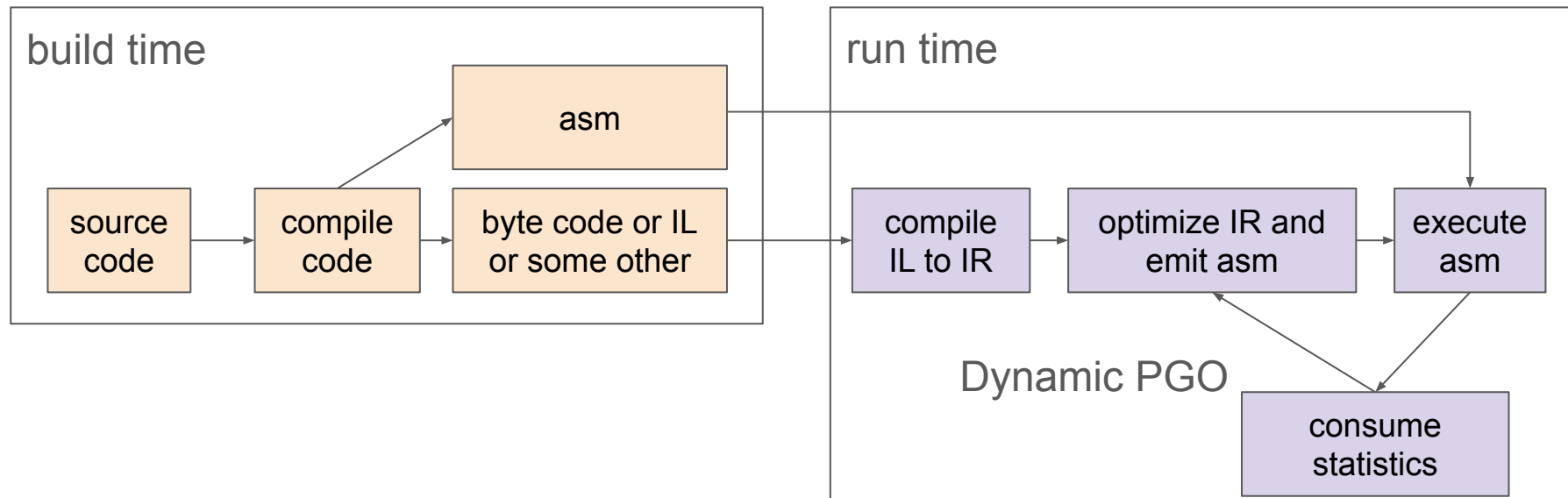
public class MyClass implements Resource {

    public MyClass() {
        Core.getGlobalContext().register(this);
    }

    @Override
    public void beforeCheckpoint(Context<? extends Resource> context) {
        /* ... */
    }

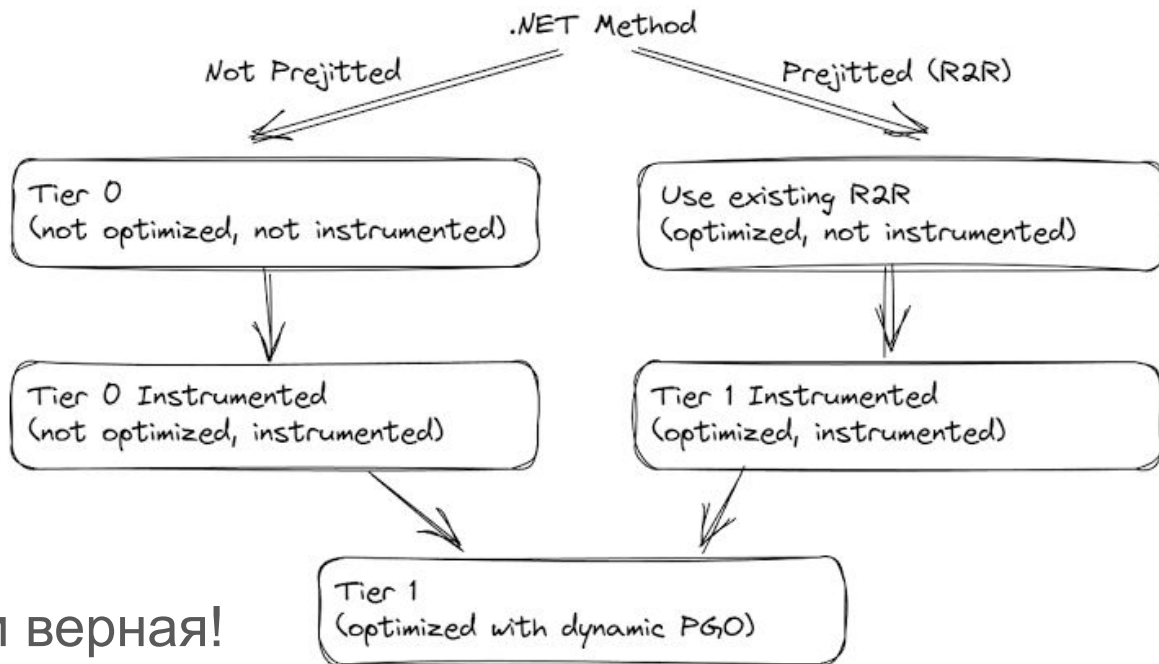
    @Override
    public void afterRestore(Context<? extends Resource> context) {
        /* ... */
    }
}
```

Мысль 3: А может что-то предкомпилировать ?



часть кода мы заранее скомпилируем и не будем использовать холодную версию

В чем выигрыш ? (Tier .NET)



она не совсем верная!

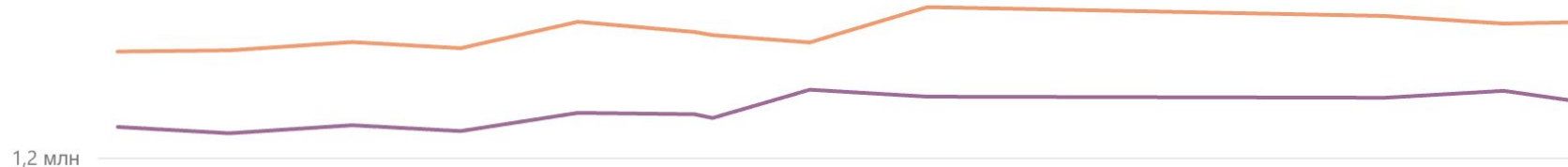
Tier0 Instrumented vs Tier1 Instrumented

```
G_M000_IG02:                ;; offset=0x0018
    ldr    x0, [fp, #0x20]
    str    x0, [fp, #0x18]
    ldr    x0, [fp, #0x18]
    movz   x1, #0x6EC0
    movk   x1, #0xBB2 LSL #16
    movk   x1, #1 LSL #32
    bl     CORINFO_HELP_CLASSPROFILE32
    ldr    x0, [fp, #0x18]
    str    x0, [fp, #0x10]
    ldr    x0, [fp, #0x10]
    movz   x11, #464
    movk   x11, #0xA4B LSL #16
    movk   x11, #1 LSL #32
    ldr    xip0, [x11]
    blr    xip0
```

```
G_M000_IG02:                ;; offset=0x0010
    mov    x0, x19
    movz   x1, #0x9CB0
    movk   x1, #0x8E5 LSL #16
    movk   x1, #1 LSL #32
    bl     CORINFO_HELP_CLASSPROFILE32
    mov    x0, x19
    movz   x11, #208
    movk   x11, #0x782 LSL #16
    movk   x11, #1 LSL #32
    ldr    xip0, [x11]
    blr    xip0
```

ВАЖНО Tier1 в итоге одинаковый !

Legend ● JsonPlatform-intel-lin-default ● JsonPlatform-intel-lin-disabled ● JsonPlatform-intel-lin-tier0-instrumented ● JsonPlatform-intel-lin-tier0-uninstrumented



```
app.MapGet("/plaintext", () => "Hello, World!");
```

<https://aka.ms/aspnet/benchmarks>





Евгений Пешков — .NET: AOT в 2022

OSR (JIT)

```
public static int Run(IWorker worker, int n)
{
    int sum = 0;
    for (int i = 0; i < n; i++) { sum += worker.Work(i); }
    return sum;
}
```

Не про отличие, а про особенность (в аот понятно, что такого нет)

OSR (JIT)

```
int Run(IWorker worker, int n)
{
    int sum = 0;
    int i = 0;
    object tmpObj = null;
    object tmpForCall = null;
    int patchCountdown = 1000;

    goto LOOP_TEST;

LOOP_BODY:
    CORINFO_HELP_COUNTPROFILE32(profileBody);

    tmpObj = worker;
    CORINFO_HELP_CLASSPROFILE32(profileClass, tmpObj);

    tmpForCall = tmpObj;
    int ret = tmpForCall.Work(i);    // косвенный вызов через blr

    sum = sum + ret;
    i = i + 1;

LOOP_TEST:
    patchCountdown = patchCountdown - 1;
    if (patchCountdown <= 0)
    {
        CORINFO_HELP_PATCHPOINT(&patchCountdown, 20);
    }

    if (i < n)
        goto LOOP_BODY;

    CORINFO_HELP_COUNTPROFILE32(profileExit);
    return sum;
}
```

Вывод

- 1) Что такое PGO, TIER, OSR и почему без PGO нельзя
- 2) AOT и JIT на разной стадии работают с PGO
- 3) Такие штуки как R2R совмещают сильные стороны AOT и JIT

Глава 2

Возможные оптимизации

Inline(все)

- GO и так все
- .NET нужен профиль

```
private static int Cheap(int x) => x + 1;
```

Ссылка: 1

```
private static int Expensive(int x)
{
    int s = 0;
    for (int i = 0; i < 50; i++){ s += x * i; }
    return s;
}
```

Ссылка: 2

```
private static int Process(int[] data, bool rareMode)
{
    int sum = 0;

    for (int i = 0; i < data.Length; i++)
    {
        if (rareMode)
        {
            sum += Expensive(data[i]);
        }
        else
        {
            sum += Cheap(data[i]);
        }
    }

    return sum;
}
```

Inline(Bce)



```
private static int Process(int[] data, bool rareMode)
{
    int sum = 0;

    if (rareMode)
    {
        for (int i = 0; i < data.Length; i++)
        {
            int x = data[i];
            int s = 0;

            for (int j = 0; j < 50; j++)
                s += x * j;

            sum += s;
        }
    }
    else
    {
        for (int i = 0; i < data.Length; i++)
            sum += data[i] + 1;
    }

    return sum;
}
```

Devirtualize (bce)

```
public interface IWorker { int Work(int x); }
```

0+1 usages

```
public sealed class Hot : IWorker { public int Work(int x) => ((x + 1) * 3) - 7; }
```

0+1 usages

```
public sealed class Cold : IWorker { public int Work(int x) => x * 5 + 11; }
```

```
public static class Program
```

```
{
```

```
    [MethodImpl(MethodImplOptions.NoInlining)]
```

3 usages

```
    public static int Run(IWorker worker, int n)
```

```
    {
```

```
        int sum = 0;
```

```
        for (int i = 0; i < n; i++) { sum += worker.Work(i); }
```

```
        return sum;
```

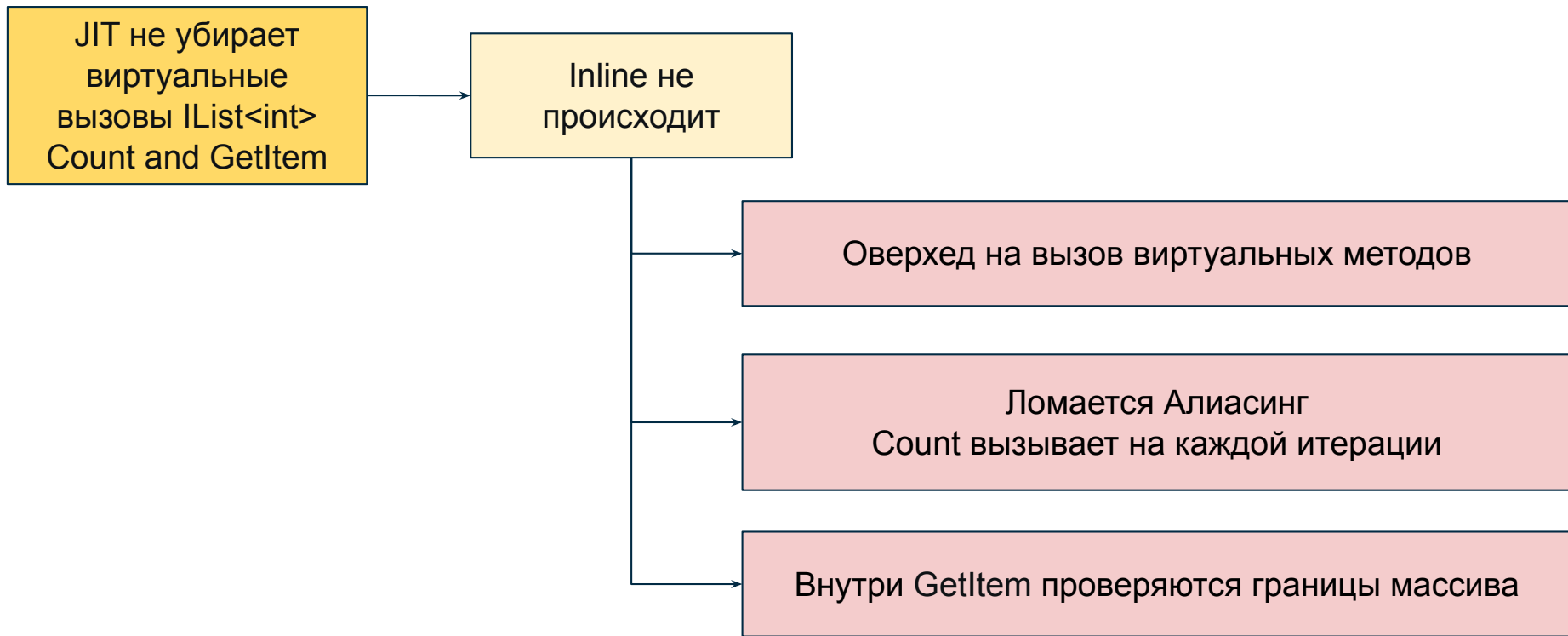
```
    }
```

inline + GDV

```
public int SumEnumerable()  
{  
    int sum = 0;  
    foreach (var item in _list)  
    {  
        sum += item;  
    }  
    return sum;  
}
```

```
public int SumForLoop()  
{  
    int sum = 0;  
    for (int i = 0; i < _list.Count; i++)  
    {  
        sum += _list[i];  
    }  
    return sum;  
}
```

Method	Runtime	Mean	Ratio
SumEnumerable	.NET 10.0	667.8 ns	0.76
SumEnumerable	.NET 9.0	881.6 ns	1.00
SumForLoop	.NET 10.0	923.8 ns	0.35
SumForLoop	.NET 9.0	2,613.5 ns	1.00



Не о каких unroll и речи быть не может...

code layout (Все, но на go не видел)

```
[MethodImpl(MethodImplOptions.NoInlining)]
static int ParseKind(int kind, int x)
{
    if (kind == 1)
        return x + 10;

    if (kind == 2)
        return x * 2;

    if (kind == 3)
        return x - 7;

    return x;
}
```

code layout (Все, но на го не видел)

```
; Tests.BinarySearch[[System.In
    push    rbp
    mov     rbp, rsp
    xor     ecx, ecx
    dec     esi
    js      short M01_L07
+   jmp     short M01_L03
M01_L00:
-   lea     eax, [rsi+rcx]
-   shr     eax, 1
-   movsxd  r8, eax
-   mov     r8d, [rdi+r8*4]
-   cmp     edx, r8d
-   jge     short M01_L03
    mov     r9d, 0FFFFFFFF
M01_L01:
    test    r9d, r9d
    je      short M01_L06
    test    r9d, r9d
    jg      short M01_L05
    lea     esi, [rax-1]
```

```
    mov     esi, [rax-1]
M01_L02:
    cmp     ecx, esi
-   jle     short M01_L00
-   jmp     short M01_L07
+   jg      short M01_L07
M01_L03:
+   lea     eax, [rsi+rcx]
+   shr     eax, 1
+   movsxd  r8, eax
+   mov     r8d, [rdi+r8*4]
    cmp     edx, r8d
-   jg      short M01_L04
-   xor     r9d, r9d
+   jl      short M01_L00
+   cmp     edx, r8d
+   jle     short M01_L04
+   mov     r9d, 1
    jmp     short M01_L01
```

Hot cold splitting (Все, но на go не видел)



```
0x1000: Entry
0x1004: cmp x, 0
0x1008: je 0x1018
```

```
0x100C: ; Fast
0x1010: mul/add
0x1014: ret
```

```
0x1018: ; Slow
0x101C: call SlowPath
0x1020: ret
```



```
; ===== HOT SECTION =====
0x1000: Entry
0x1004: cmp x, 0
0x1008: je 0x2000 ; прыжок в cold
```

```
0x100C: ; Fast
0x1010: mul/add
0x1014: ret
```

```
; ===== COLD SECTION =====
0x2000: ; Slow
0x2004: call SlowPath
0x2008: ret
```

Java uncommon trap

```
static int work(Number n) {  
    if (n instanceof Integer i) {  
        return i + 1;           // hot path  
    }  
  
    // редкий путь  
    return slowConvert(n);  
}
```

.NET IsInstanceOfClass

```
private A _obj = new C();

[Benchmark]
public bool IsInstanceOf() => _obj is B;

public class A { }
public class B : A { }
public class C : B { }
```

.NET IsInstanceOfClass



```
bool IsInstanceOf()  
{  
    object obj = this._field;  
  
    object result = CastHelpers.IsInstanceOfClass(typeof(B).MethodTable, obj);  
  
    return result != null;  
}  
  
bool IsInstanceOf()  
{  
    object obj = this._field;  
  
    if (obj == null)  
        return false;  
  
    if (obj.MethodTable == MethodTableOf<C>())  
        return true;  
  
    object result = CastHelpers.IsInstanceOfClass(typeof(B).MethodTable, obj);  
  
    return result != null;  
}
```

PGO + escape analysis (.NET)



```
static int Sum(IEnumerable<int> e)
{
    int r = 0;
    foreach (int i in e)
        r += i;
    return r;
}
```



```
static int Sum(IEnumerable<int> e)
{
    int r = 0;
    if (e is List<int> list)
    {
        var en = list.GetEnumerator();
        while (en.MoveNext())
            r += en.Current;
        return r;
    }

    // SLOW PATH (оригинальные блоки):
    foreach (int i in e)
        r += i;
    return r;
}
```

Резюмируя

- 1) Go инлайн + девиртуализация
- 2) .NET инлайн + девиртуализация + type-test/cast specialization + hot/cold/branch layout
- 3) Java инлайн + девиртуализация + speculative optimization + branch/loop optimizations + code layout

Глава 3.

В чем разница в PGO на практике между
языками ?

Начнем с примера

```
public interface IWorker { int Work(int x); }
```

0+1 usages

```
public sealed class Hot : IWorker { public int Work(int x) => ((x + 1) * 3) - 7; }
```

0+1 usages

```
public sealed class Cold : IWorker { public int Work(int x) => x * 5 + 11; }
```

```
public static class Program
```

```
{
```

```
    [MethodImpl(MethodImplOptions.NoInlining)]
```

3 usages

```
    public static int Run(IWorker worker, int n)
```

```
    {
```

```
        int sum = 0;
```

```
        for (int i = 0; i < n; i++) { sum += worker.Work(i); }
```

```
        return sum;
```

```
    }
```

Результат сборки .NET

; Assembly listing for method Program:Run(IWorker,int):int (FullO
; Emitting BLENDED_CODE for generic ARM64 on Apple
; FullOpts code
; optimized code
; fp based frame
; partially interruptible
; No PGO data



```
function Run(worker: IWorker, count: int) -> int:
    if count <= 0:
        return 0

    sum = 0
    for i = 0 to count - 1:
        result = worker.SomeMethod(i) // вызов метода по фиксированному адресу 0x16860038
        sum += result

    return sum
```

GO

```
type Worker interface { Work(x int) int }
type Hot struct{}
type Cold struct{}
```

```
func (Hot) Work(x int) int {
    y := x + 1
    y *= 3
    return y - 7
}
```

```
func (Cold) Work(x int) int {
    return x*5 + 11
}
```

```
//go:noinline
```

```
func runLoop(w Worker, n int) int {
    sum := 0
    for i := 0; i < n; i++ {
        sum += w.Work(i)
    }
    return sum
}
```



```
func runLoop(obj *SomeStruct, arg uintptr, count int64) int64 {
    var i int64 = 0
    var acc int64 = 0

    for i < count {
        fn := *(*func(uintptr, int64) int64)(unsafe.Pointer(uintptr(unsafe.Pointer(obj)) + 0x18))
        acc += fn(arg, i)
        i++
    }

    return acc
}
```

Запустим Dotnet



```
int Method(object? obj, int n)
{
    int sum = 0;

    for (int i = 0; i < n; i++)
    {
        if (obj != null && obj.MethodTable == CONST_TYPE_HANDLE)
            sum += 3 * (i + 1) - 7;
        else
            sum += CallHelperOrVirtualMethod(obj, i);
    }

    return sum;
}
```

Запустим Go



```
func runLoop(obj *SomeStruct, arg uintptr, count int64) int64 {  
    var i int64 = 0  
    var acc int64 = 0  
  
    for i < count {  
        fn := *(*func(uintptr, int64) int64)(unsafe.Pointer(uintptr(unsafe.Pointer(obj)) + 0x18))  
        acc += fn(arg, i)  
        i++  
    }  
  
    return acc  
}
```

```
go build -o app . && GODEBUG=asyncpreemptoff=1 ./app profile >/dev/null
```

```
&& go build -a -pgo=off -o app-off .
```

```
&& go build -a -pgo=cpu.pprof -o app-on .
```

```
&& echo '=== OFF ===' && go tool objdump -s 'main\.runLoop|main\.run|main\.Hot\.Work|main\.Cold\.Work' ./app-off
```

```
&& echo '=== ON ===' && go tool objdump -s 'main\.runLoop|main\.run|main\.Hot\.Work|main\.Cold\.Work' ./app-on
```

Запустим Go

```
func (Hot) Work(x int) int {  
    y := x + 1  
    a := y*3 - 7  
    b := a + x*5 + 11  
    c := b ^ (a + x)  
    d := c + b - a + x  
    if d&1 == 0 {  
        d += a & 7  
    } else {  
        d -= b & 3  
    }  
    e := d*3 - c + 17  
    if e < 0 {  
        e = -e  
    }  
    return e + a  
}
```

```
func (Cold) Work(x int) int {  
    y := x*5 + 11  
    z := y + x*2 - 3  
    if z&1 == 0 {  
        z += y  
    }  
    return z  
}
```

```
func workload(a, b, rounds int) int {  
    total := 0  
    i := 0  
  
    for i < a+b {  
        worker := pickWorker(i, a)  
  
        j := 0  
        for j < rounds {  
            if worker is Hot {  
  
                total += // заинлайненный Hot.Work(j)  
            } else {  
                total += worker.Work(j)  
            }  
  
            j++  
        }  
  
        i++  
    }  
  
    return total  
}
```

а теперь изменилось и даже результат на этом примере

Различия

.NET в любом случае сделал оптимизацию, а GO потребовал увеличить тело метода

Почему так ???

Глава 3.

Почему мы увеличили тело метода на GO?

Типы сбора статистики

1) Instrumentation-based

```
int Run(IWorker worker, int n)
{
    int sum = 0;
    int i = 0;
    object tmpObj = null;
    object tmpForCall = null;
    int patchCountdown = 1000;

    goto LOOP_TEST;

LOOP_BODY:
    CORINFO_HELP_COUNTPROFILE32(profileBody);

    tmpObj = worker;
    CORINFO_HELP_CLASSPROFILE32(profileClass, tmpObj);

    tmpForCall = tmpObj;
    int ret = tmpForCall.Work(i);    // косвенный вызов через blr

    sum = sum + ret;
    i = i + 1;

LOOP_TEST:
    patchCountdown = patchCountdown - 1;
    if (patchCountdown <= 0)
    {
        CORINFO_HELP_PATCHPOINT(&patchCountdown, 20);
    }

    if (i < n)
        goto LOOP_BODY;

    CORINFO_HELP_COUNTPROFILE32(profileExit);
    return sum;
}
```



Где PGO собирается, вставляется и применяется ?

```
public static int Run(IWorker worker, int n)
{
    int sum = 0;
    for (int i = 0; i < n; i++) { sum += worker.Work(i); }
    return sum;
}
```

```
* STORE_LCL_VAR int    V02 loc0
  \--* ADD          int
    +--* LCL_VAR    int    V02 loc0
      \--* CALLV stub int   IWorker:Work(int):int:this
        +--* LCL_VAR  ref    V00 arg0
          \--* LCL_VAR  int    V03 loc1
```

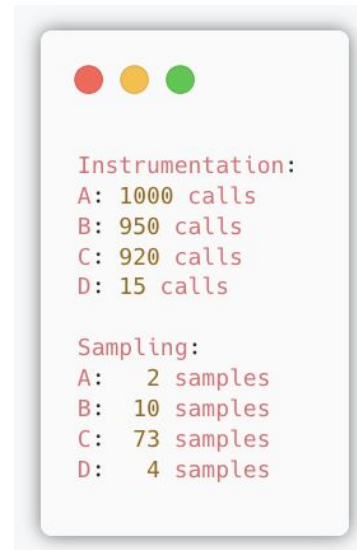
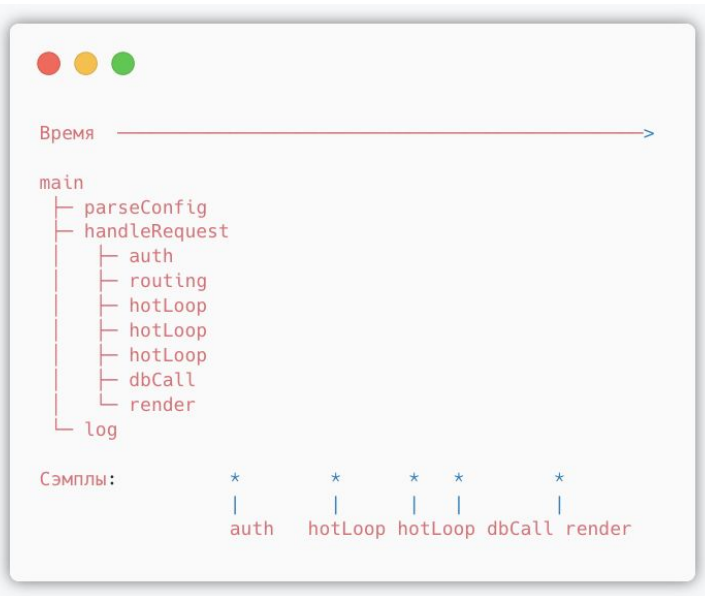
Где PGO собирается, вставляется и применяется ?

```
public static int Run(IWorker worker, int n)
{
    int sum = 0;
    for (int i = 0; i < n; i++) { sum += worker.Work(i); }
    return sum;
}
```

```
STMT
├─ STORE_LCL_VAR int V02 loc0
│   └─ ADD int
│       ├── LCL_VAR int V02 loc0
│       └─ CALLV stub int IWorker:Work(int):int:this
│           ├── COMMA ref [this]
│           │   ├── STORE_LCL_VAR ref V05 tmp1
│           │   │   └─ LCL_VAR ref V00 arg0
│           │   └─ COMMA ref
│           │       ├── CALL help void CORINFO_HELP_CLASSPROFILE32
│           │       │   ├── LCL_VAR ref V05 tmp1 [arg0]
│           │       │   └─ CNS_INT long 0x10b5bd920 [arg1]
│           │       └─ LCL_VAR ref V05 tmp1
│           └─ LCL_VAR int V03 loc1 [arg1]
```

Типы сбора статистики

- 1) Instrumentation-based
- 2) Sampling-based



Sampling-based

Go CPU profiling работает так:

- по умолчанию `runtime/pprof.StartCPUProfile` ставит частоту 100 Hz
- то есть примерно 100 samples в секунду
- это примерно один sample каждые 10 ms но не для всех потоков сразу



```
if sig == _SIGPROF {  
    if !delayedSignal && validSIGPROF(mp, c) {  
        sigprof(c.sigpc(), c.sigsp(), c.siglr(), gp, mp)  
    }  
    return  
}
```

Данные разрезены



CFG:



Реальное выполнение:

B1 = 1 000 000
B2 = 990 000
B3 = 10 000
B4 = 1 000 000

Sampling (20 sample'ов):

B1 = 2
B2 = 8
B3 = 0
B4 = 10

Как .NET инструментирует?

- Быстро
- Не потокобезопасно (race condition) -> Потеря обновлений при многопоточном доступе



Как .NET инструментирует?



```
Interlocked.Increment(ref counter);
```

- Потокбезопасно
- Точная семантика "прибавь 1"
- Медленно при высокой конкуренции, узкое место

Как .NET инструментирует?

- Потокбезопасно
- Почти такая же производительность, как у ++
- Приблизленно верно, отклонение < 1%



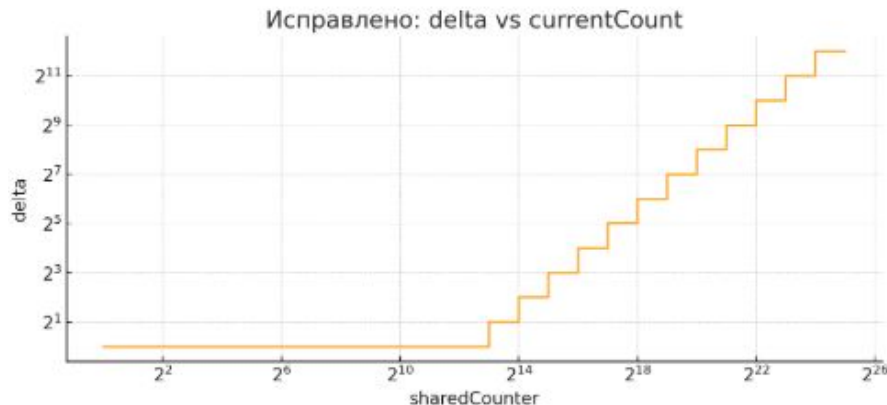
```
static void Count(ref uint sharedCounter)
{
    uint currentCount = sharedCounter, delta = 1;
    if (currentCount > 0)
    {
        int logCount = 31 - (int)uint.LeadingZeroCount(currentCount);
        if (logCount >= 13)
        {
            delta = 1u << (logCount - 12);
            uint random = (uint)Random.Shared.NextInt64(0, uint.MaxValue + 1L);
            if ((random & (delta - 1)) != 0) return;
        }
    }

    Interlocked.Add(ref sharedCounter, delta);
}
```

delta vs sharedCounter
до 8192 (2^{13}) — шаг
ровно 1, дальше
удваивается каждые
 $2\times$

```
static void Count(ref uint sharedCounter)
{
    uint currentCount = sharedCounter, delta = 1;
    if (currentCount > 0)
    {
        int logCount = 31 - (int)uint.LeadingZeroCount(currentCount);
        if (logCount >= 13)
        {
            delta = 1u << (logCount - 12);
            uint random = (uint)Random.Shared.NextInt64(0, uint.MaxValue + 1L);
            if ((random & (delta - 1)) != 0) return;
        }
    }

    Interlocked.Add(ref sharedCounter, delta);
}
```

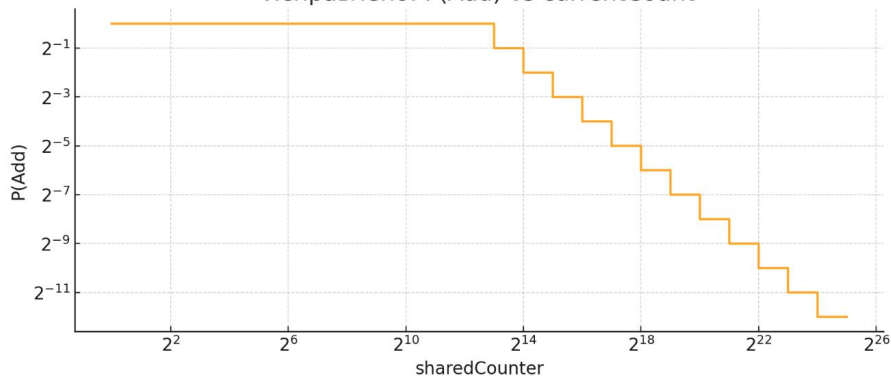


P(Add)
vs sharedCounter
вероятность вызова
Interlocked.Add
падает как $1/\delta$

```
static void Count(ref uint sharedCounter)
{
    uint currentCount = sharedCounter, delta = 1;
    if (currentCount > 0)
    {
        int logCount = 31 - (int)uint.LeadingZeroCount(currentCount);
        if (logCount >= 13)
        {
            delta = 1u << (logCount - 12);
            uint random = (uint)Random.Shared.NextInt64(0, uint.MaxValue + 1L);
            if ((random & (delta - 1)) != 0) return;
        }
    }

    Interlocked.Add(ref sharedCounter, delta);
}
```

Исправлено: P(Add) vs currentCount



Как .NET инструментирует?



```
while (true)
{
    Run("Interlock", _ => {
        for (int i = 0; i < ItersPerThread; i++) Interlocked.Increment(ref counter);
    });
    Run("Racy", _ => {
        for (int i = 0; i < ItersPerThread; i++) counter++;
    });
    Run("Scalable", _ => {
        for (int i = 0; i < ItersPerThread; i++) Count(ref counter);
    });
}
```

Как .NET инструментирует?



Попытка 1

Interlock => Expected: 1 200 000 000, Actual: 1 200 000 000, Elapsed: 108030,9295ms
Racy => Expected: 1 200 000 000, Actual: 148 620 908, Elapsed: 283,6168ms
Scalable => Expected: 1 200 000 000, Actual: 1 200 886 164, Elapsed: 557,9463ms

Попытка 2

Interlock => Expected: 1 200 000 000, Actual: 1 200 000 000, Elapsed: 88186,767ms
Racy => Expected: 1 200 000 000, Actual: 179 682 457, Elapsed: 321,0945ms
Scalable => Expected: 1 200 000 000, Actual: 1 193 279 524, Elapsed: 520,8084ms

Попытка 3

Interlock => Expected: 1 200 000 000, Actual: 1 200 000 000, Elapsed: 94467,3402ms
Racy => Expected: 1 200 000 000, Actual: 147 883 941, Elapsed: 284,1835ms
Scalable => Expected: 1 200 000 000, Actual: 1 185 415 649, Elapsed: 427,7372ms

Куда ставить инструкции ?

$B \rightarrow C = 80$

$D \rightarrow E = 20$

$F \rightarrow E = 75$



Глава 4.

Как JIT пришел к результату

Рассмотрим процесс оптимизации с OSR

```
public interface IWorker { int Work(int x); }
```

0+1 usages

```
public sealed class Hot : IWorker { public int Work(int x) => ((x + 1) * 3) - 7; }
```

0+1 usages

```
public sealed class Cold : IWorker { public int Work(int x) => x * 5 + 11; }
```

```
public static class Program
```

```
{
```

```
    [MethodImpl(MethodImplOptions.NoInlining)]
```

3 usages

```
    public static int Run(IWorker worker, int n)
```

```
    {
```

```
        int sum = 0;
```

```
        for (int i = 0; i < n; i++) { sum += worker.Work(i); }
```

```
        return sum;
```

```
    }
```

```
dotnet build -c Release -f net10.0 &&
```

```
COMPlus_TieredCompilation=1
```

```
COMPlus_TieredPGO=1
```

```
COMPlus_ReadyToRun=0
```

```
COMPlus_TC_QuickJit=1
```

```
COMPlus_TC_QuickJitForLoops=1
```

```
COMPlus_TC_CallCountingDelayMs=0
```

```
COMPlus_JitDisasm="*Test:GetLong"
```

```
./bin/Release/net10.0/Inline profile
```

Instrumented Tier0

- все проливается через стек
- в теле цикла есть явные вызовы профилирования
- точка, где рантайм может переключить выполнение на OSR-версию
- Нет оптимизаций

```
int Run(IWorker worker, int n)
{
    int sum = 0;
    int i = 0;
    object tmpObj = null;
    object tmpForCall = null;
    int patchCountdown = 1000;

    goto LOOP_TEST;

LOOP_BODY:
    CORINFO_HELP_COUNTPROFILE32(profileBody);

    tmpObj = worker;
    CORINFO_HELP_CLASSPROFILE32(profileClass, tmpObj);

    tmpForCall = tmpObj;
    int ret = tmpForCall.Work(i);    // косвенный вызов через blr

    sum = sum + ret;
    i = i + 1;

LOOP_TEST:
    patchCountdown = patchCountdown - 1;
    if (patchCountdown <= 0)
    {
        CORINFO_HELP_PATCHPOINT(&patchCountdown, 20);
    }

    if (i < n)
        goto LOOP_BODY;

    CORINFO_HELP_COUNTPROFILE32(profileExit);
    return sum;
}
```

Tier1-OSR

```
int Run_Tier1_OSR(IWorker worker, int n, int i, int sum)
{
    if (i >= n)
        return sum;

    // fast path: ожидаемый hot type
    if (worker != null && worker.MethodTable == Hot_MethodTable)
    {
        do
        {
            i = i + 1;
            int value = (i * 3) - 7;    // inlined Hot.Work(i)
            sum = sum + value;
        }
        while (i < n);

        return sum;
    }

    // fallback path
    while (i < n)
    {
        int value;

        if (worker.MethodTable == Hot_MethodTable)
        {
            value = ((i + 1) * 3) - 7;
        }
        else
        {
            value = worker.Work(i);    // indirect dispatch fallback
        }

        sum = sum + value;
        i = i + 1;
    }

    return sum;
}
```

Tier1

```
G_M000_IG05:                ;; offset=0x004C
                                add     w22, w22, #1
                                mov     w0, #3
                                mul     w0, w22, w0
                                sub     w25, w0, #7
                                add     w21, w21, w25
                                cmp     w22, w19
                                blt     G_M000_IG05
```

Самая горячая часть метода супер
компактная

```
int Run_Tier1(IWorker worker, int n)
{
    int sum = 0;
    int i = 0;

    if (n <= 0)
        return sum;

    // fast path для hot concrete type
    if (worker != null && worker.MethodTable == Hot_MethodTable)
    {
        do
        {
            i = i + 1;
            int value = (i * 3) - 7; // inlined Hot.Work(i)
            sum = sum + value;
        }
        while (i < n);

        return sum;
    }

    // general path
    while (i < n)
    {
        int value;

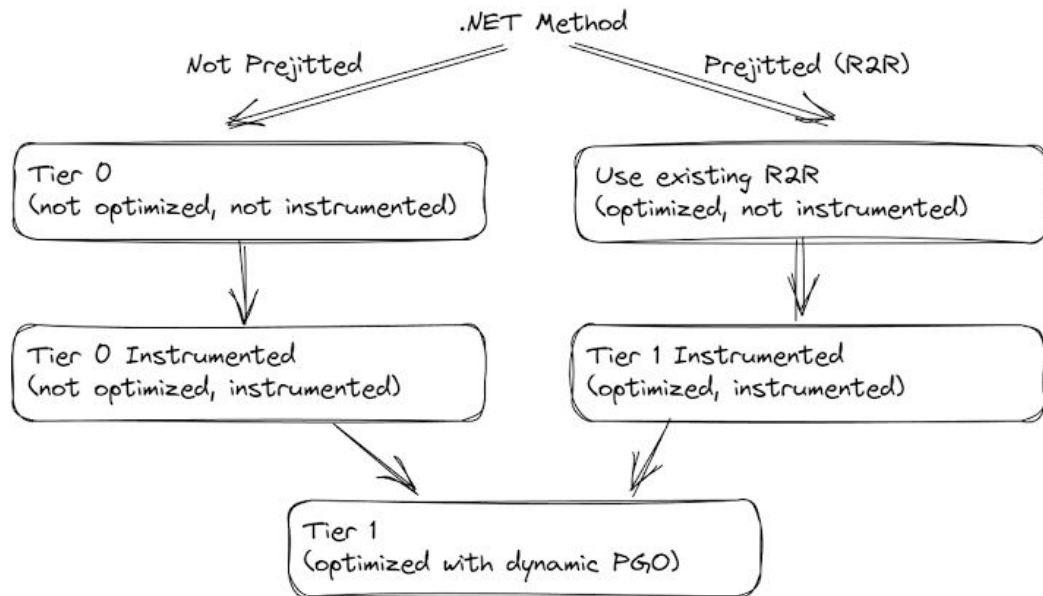
        if (worker.MethodTable == Hot_MethodTable)
        {
            value = ((i + 1) * 3) - 7;
        }
        else
        {
            value = worker.Work(i); // indirect fallback
        }

        sum = sum + value;
        i = i + 1;
    }

    return sum;
}
```

Ключевое

- 1) Прогрев слабая сторона
- 2) Только Java может адаптировать под изменяющийся профиль и то не вечно



Глава 4

Куда все движется ?

1) JIT платформы пытаются и на Tier 0 срезать углы

```
static void Test()
{
    long gc = GC.GetAllocatedBytesForCurrentThread();
    for (int i = 0; i < 100; i++)
    {
        ThrowIfNull(i);
    }
    gc = GC.GetAllocatedBytesForCurrentThread() - gc;

    Console.WriteLine(gc);
    Thread.Sleep(1000);
}
```

2) Активно развивается runtime to build time

- .NET внедряет source gen вместо reflection

```
using Pure.DI;  
  
var composition = new Composition();  
var service = composition.Root<int>();
```

```
DI.Setup(nameof(Composition))  
    .Root<Service<TTS>>( name: "Root");
```

✖ 2 usages

```
class Dependency;
```

✖ 3 usages ✖ 1 exposing API

```
class Service<T>(Dependency dependency)  
    where T: struct;
```

```
{  
    _rootM03D14di = baseComposition._rootM03D14d  
}
```

```
#region Composition Roots  
#if NETSTANDARD2_0_OR_GREATER || NETCOREAPP ||  
[global::System.Diagnostics.Contracts.Pure]  
#endif
```

❖ IL code

```
public Service<T> Root<T>()  
    where T: struct  
{  
    return new Service<T>(new Dependency());  
}  
#endregion
```

```
}
```

```
#pragma warning restore
```

2) Активно развивается runtime to build time

- Quarkus Java (переписывает байт код)

```
@ApplicationScoped
public class GreetingService {
    public String hello() {
        return "hello";
    }
}

@ApplicationScoped
public class UseCase {
    @Inject GreetingService service;

    public String run() {
        return service.hello();
    }
}
```

```
class GreetingService_Bean {
    GreetingService create() {
        return new GreetingService();
    }
}

class UseCase_Bean {
    UseCase create(GreetingService service) {
        UseCase u = new UseCase();
        u.service = service;
        return u;
    }
}
```

3) Активно развивается aot

И Java и .NET вкладываются в
АОТ последние годы

Feature	Fully Supported	Partially Supported	Not Supported
gRPC	✓		
Minimal APIs		✓	
MVC			✗
Blazor Server			✗
SignalR		✓	
JWT Authentication	✓		
Other Authentication			✗
CORS	✓		
HealthChecks	✓		
HttpLogging	✓		
Localization	✓		
OutputCaching	✓		
RateLimiting	✓		
RequestDecompression	✓		
ResponseCaching	✓		
ResponseCompression	✓		
Rewrite	✓		
Session			✗
Spa			✗
StaticFiles	✓		
WebSockets	✓		

3) Развиваются попытки переиспользовать профиль

```
const char* Compiler::compGetPgoSourceName() const
{
    switch (fgPgoSource)
    {
        case ICorJitInfo::PgoSource::Static:
            return "Static PGO";
        case ICorJitInfo::PgoSource::Dynamic:
            return "Dynamic PGO";
        case ICorJitInfo::PgoSource::Blend:
            return "Blended PGO";
        case ICorJitInfo::PgoSource::Text:
            return "Textual PGO";
        case ICorJitInfo::PgoSource::Sampling:
            return "Sample-based PGO";
        case ICorJitInfo::PgoSource::IBC:
            return "Classic IBC";
        case ICorJitInfo::PgoSource::Synthesis:
            return "Synthesized PGO";
        default:
            return "Unknown PGO";
    }
}
```

Почему ?

Время старта (простои серверов)

Вывод

- 1) И .NET и Go и Java используют PGO (все современные языки используют PGO)
- 2) Различия
 - a) Время применения оптимизации
 - b) способ снятия профиля
 - c) кол-во возможных оптимизаций
 - d) реагирование на изменение профиля после оптимизации
- 3) Импакт (в порядке важности)
 - a) Java, .NET, Go